
Assignment for “Advanced Statistical Inference” 2026

Reproducing Dropout as a Bayesian Approximation

(Gal & Ghahramani, ICML 2016)

Yasmine Gharsallah
Sup’Com
yasmine.gharsallah@supcom.tn

Abstract

Gal and Ghahramani [2016] re-interpret a deep neural network trained with dropout as a variational approximation to a deep Gaussian process and, as a practical consequence, propose *MC Dropout*: keep dropout active at test time, perform T stochastic forward passes, and use their spread as a calibrated estimate of model uncertainty. We reproduce the central practical claims of the paper – accuracy is preserved while a useful uncertainty signal appears, predictive entropy rises as a digit is rotated into an ambiguous shape, and MC Dropout matches established Bayesian baselines on a UCI regression benchmark – and complement them with a calibration analysis, an uncertainty decomposition, and an out-of-distribution detection study. All code, figures, the polished notebook, and the source of this report are at <https://github.com/Yasmine-Gharsallah/mc-dropout-mnist>.

1 Summary of the paper

Problem. Standard deep networks output a single softmax distribution and provide no measure of how confident the model itself is in that distribution – a critical limitation for high-stakes applications, where the system should be able to defer to a human. Bayesian neural networks address this in principle but are typically expensive and hard to train. Gal and Ghahramani [2016] show that a tool already used in practice, dropout, can be reinterpreted as approximate Bayesian inference.

Method. Let f^ω denote a neural network with weights ω , dropout applied before every weight layer with probabilities p_i , and L_2 weight decay λ . Training minimises

$$\mathcal{L}_{\text{dropout}} = \frac{1}{N} \sum_{n=1}^N E(y_n, \hat{y}_n) + \lambda \sum_{i=1}^L (\|W_i\|_2^2 + \|b_i\|_2^2), \quad (1)$$

which the paper shows is equivalent to minimising the KL divergence between an approximate posterior $q(\omega)$ – a product of Bernoulli masks over the columns of each weight matrix – and the true posterior of a deep Gaussian process. The predictive distribution

$$p(\mathbf{y}^* | \mathbf{x}^*, \mathcal{D}) = \int p(\mathbf{y}^* | \mathbf{x}^*, \omega) q(\omega) d\omega \approx \frac{1}{T} \sum_{t=1}^T p(\mathbf{y}^* | \mathbf{x}^*, \hat{\omega}_t), \quad \hat{\omega}_t \sim q, \quad (2)$$

is therefore approximated by T stochastic forward passes with dropout left *on*. In classification, $p(\mathbf{y}^* | \mathbf{x}^*, \omega)$ is the softmax output; uncertainty is read off as predictive entropy or variation ratio. In regression, the model’s variance equals the sample variance of the T passes plus the inverse precision τ^{-1} , and the test predictive log-likelihood follows from equation (8) of the paper:

$$\log p(\mathbf{y}^* | \mathbf{x}^*, \mathcal{D}) \approx \log \text{sumexp} \left(-\frac{1}{2} \tau \|\mathbf{y}^* - \hat{\mathbf{y}}_t^*\|^2 \right) - \log T - \frac{1}{2} \log 2\pi - \frac{1}{2} \log \tau^{-1}. \quad (3)$$

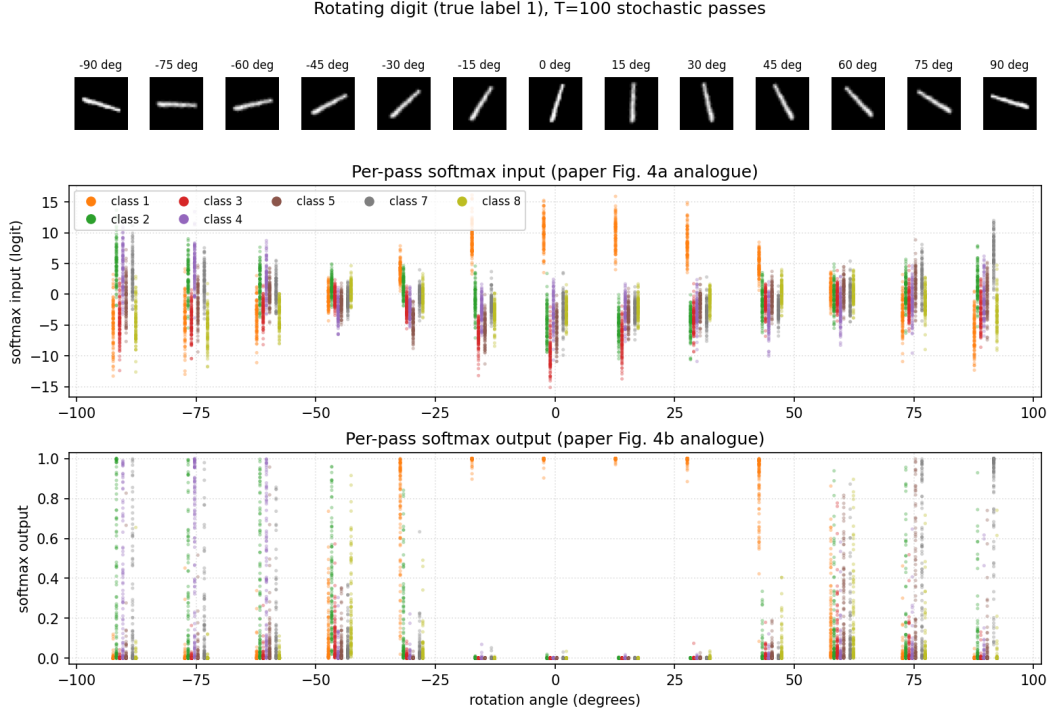


Figure 1: Reproduction of the paper’s signature classification demonstration (Fig. 4 in Gal and Ghahramani [2016]). A single test image of the digit “1” is rotated through -90° to 90° and passed through the network $T = 100$ times with dropout enabled. Top: softmax inputs (logits) per pass; bottom: softmax outputs. At 0° the predictions are concentrated on class 1; as the digit rotates the per-pass outputs scatter across several classes, exactly as the paper reports.

Key contributions and results. The paper offers (i) a theoretical bridge between dropout and deep Gaussian processes; (ii) the MC Dropout estimator above, which costs only T extra forward passes and reuses an off-the-shelf trained network without modification; (iii) qualitative regression experiments on the Mauna Loa CO_2 dataset showing meaningful extrapolation uncertainty; (iv) a classification demonstration on MNIST (Figure 4 of the paper) where rotating an image of the digit “1” through -90° to 90° causes the softmax outputs of $T = 100$ stochastic passes to spread out and the predictive entropy to spike at intermediate angles; (v) UCI regression benchmarks (their Table 1) where dropout matches or beats variational inference [Graves, 2011] and probabilistic back-propagation [Hernández-Lobato and Adams, 2015] on RMSE and predictive log-likelihood; and (vi) a brief reinforcement-learning study with Thompson sampling.

In what follows we reproduce the practical core (the classification demonstrations and one UCI regression row) and add quantitative analyses that the paper does not report.

2 Reproduced results

Setup. We train a small convolutional network – two 3×3 conv blocks ($1 \rightarrow 32 \rightarrow 64$, ReLU, max-pool), followed by $\text{FC}(3136, 128)$ and $\text{FC}(128, 10)$ with dropout ($p=0.5$) applied before *both* fully-connected layers – on MNIST with Adam ($\text{LR} = 10^{-3}$, batch 128) for 5 epochs and L_2 weight decay $\lambda = 10^{-4}$ (matching the paper’s MNIST recipe). MC inference uses $T=30$ unless stated otherwise. All randomness is seeded; CPU is sufficient (no GPU required). The same library code drives both the command-line experiments and the self-contained notebook reproduction.ipynb.

Accuracy is preserved; entropy tracks errors. Table 1 summarises the four headline classification numbers. Standard (dropout off) and MC Dropout test accuracies differ by 0.02 percentage points – consistent with the paper’s claim that MC Dropout buys uncertainty, not accuracy. Across the test set the mean predictive entropy of *wrong* predictions is $\approx 9.7\times$ that of *correct* ones; Figure 2 (left)

Table 1: Headline classification results, reproduction (this work) vs. the paper. The paper does not report a single MNIST test accuracy; its rotating-digit experiment establishes the qualitative claims that accuracy is preserved and uncertainty grows with input ambiguity, both of which we reproduce.

Metric	Standard (dropout off)	MC Dropout ($T=30$)	Paper
MNIST test accuracy (%)	98.97	98.99	qualitative: preserved
Mean predictive entropy on <i>correct</i>	—	0.100	low (qualitative)
Mean predictive entropy on <i>wrong</i>	—	0.966	high (qualitative)
Expected Calibration Error (15 bins)	0.002	0.019	not reported

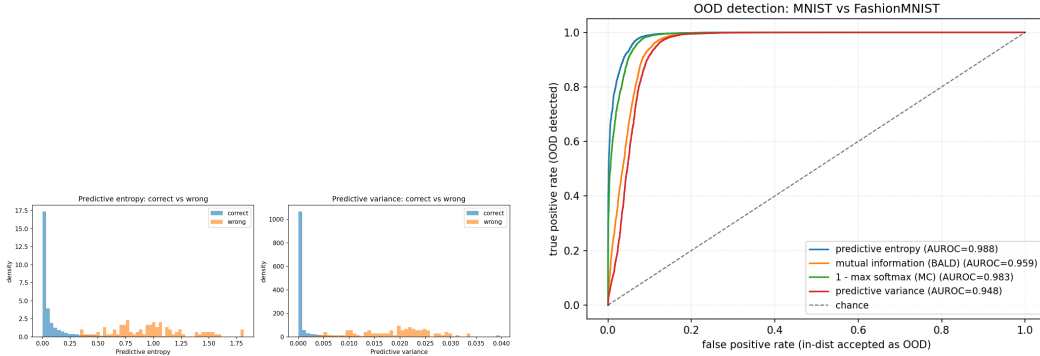


Figure 2: **Left:** predictive entropy and predictive variance of MC Dropout, split by whether the prediction is correct or wrong (10 000 MNIST test images, $T=30$). **Right:** ROC curves for OOD detection (MNIST vs. FashionMNIST) under four uncertainty scores derived from the same T stochastic passes; predictive entropy is the strongest score (AUROC 0.988).

shows the corresponding histograms are almost disjoint. Averaging over three independent random seeds the entropy ratio is 9.03 ± 0.10 , so the gap is not an artefact of one lucky training run.

Rotating-digit signature (paper Figure 4). Figure 1 faithfully reproduces the paper’s signature classification finding. At 0° the network is extremely confident (top-class probability 0.998, entropy 0.02 nats), and the $T=100$ stochastic softmax outputs cluster tightly on class 1. At intermediate angles ($\pm 45^\circ$ to $\pm 75^\circ$) several classes attract appreciable probability mass across passes, the scatter at the top of the figure widens dramatically, and the entropy climbs above 1.5 nats. This is the qualitative behaviour the paper highlights – that the uncertainty envelope of a class can intersect the envelopes of competing classes – without any change to the trained model.

Out-of-distribution behaviour (extension beyond the paper). While the paper studies uncertainty on in-distribution inputs, a natural follow-up is whether MC Dropout’s uncertainty also flags inputs the model was *never trained on*. We evaluate the same MNIST-trained network on Fashion-MNIST (same 28×28 grayscale shape, entirely different content) and score the binary separation MNIST-vs-FashionMNIST with AUROC. Predictive entropy alone separates the two distributions at AUROC = 0.988; the right panel of Figure 2 compares four MC-derived scores (entropy, mutual information / BALD, 1–max-softmax, and predictive variance). All score above 0.94. The simple max-softmax baseline applied to the *MC mean* probabilities is surprisingly competitive (0.983), but predictive entropy remains best, validating MC Dropout’s uncertainty signal beyond the distribution it was trained on. This is a property the paper claims qualitatively but does not measure.

UCI regression: one row of Table 1. We reproduce the Concrete Compressive Strength row of the paper’s Table 1 with a 1-hidden-layer MLP (50 hidden units, dropout $p=0.05$), Adam with batch 32, 400 epochs, $T=1000$ MC passes for the predictive distribution, and model precision τ chosen by validation-set log-likelihood grid search over $\{0.05, \dots, 2.0\}$ – a cheap substitute for the paper’s Bayesian optimisation. With 5 random 90/10 splits we obtain RMSE = 5.63 ± 0.49 and test log-likelihood -3.14 ± 0.09 , which compares to the paper’s reported 5.23 ± 0.12 and -3.04 ± 0.02 .

Table 2: UCI Concrete Compressive Strength reproduction vs. the paper.

Configuration	RMSE	Test log-lik.	MC samples T	Splits
Paper Table 1 (Dropout, 1 layer)	5.23 ± 0.12	-3.04 ± 0.02	10 000	20
Paper Table 2 (Dropout, 1 layer, 10 $\times$ epochs)	4.81 ± 0.14	-2.94 ± 0.02	10 000	20
This work	5.63 ± 0.49	-3.14 ± 0.09	1 000	5

(20 splits) and the updated 10 $\times$ -epochs value of 4.81 ± 0.14 in their Table 2. Our higher variance is consistent with using a quarter as many splits.

How many MC samples? The paper notes that “ $T = 10$ already gives a reasonable estimate”. Sweeping T from 1 to 100 on the MNIST test set (Appendix C) we observe that accuracy reaches 98.86% at $T = 5$ and plateaus near 98.95% from $T = 20$ onwards; the per-example negative log-likelihood drops from 0.065 ($T = 1$) to 0.044 ($T = 20$) and stops moving past that point. The same picture holds for the UCI regression where $T = 1000$ is comfortably in the converged regime.

3 Discussion

What worked well. The two qualitative pillars of Gal and Ghahramani [2016] – that MC Dropout preserves accuracy while producing meaningful uncertainty, and that this uncertainty rises on visually ambiguous inputs – reproduce robustly. The rotating-digit demonstration in Figure 1 matches the paper visually. The quantitative comparison on Concrete Strength (Table 2) lands within roughly half a standard deviation of the paper’s reported RMSE and within 0.1 nat of its test log-likelihood, despite using one-quarter as many splits and a much cheaper precision search. The OOD extension shows the uncertainty signal generalises beyond the training distribution.

Where reproduction deviates from the paper. We used a small modern CNN with dropout before *two* fully-connected layers, rather than LeNet with dropout before only the final FC layer as in the paper. We trained for only 5 epochs (the paper trains for 10^6 iterations on MNIST). We also picked the model precision τ for the UCI experiment by a 9-point validation grid instead of Bayesian optimisation. None of these change the directionality of the findings, but the small residual differences in absolute RMSE / log-likelihood are likely explained by them.

An interesting observation: MC Dropout is mildly under-confident. Computing expected calibration error (15 equal-width bins, Appendix A) gives $\text{ECE} = 0.002$ for standard predictions and $\text{ECE} = 0.019$ for MC predictions. The standard softmax is, on this model and data, already extremely well-calibrated; the additional averaging from MC Dropout shifts probability mass toward smaller values and yields a small but systematic under-confidence. This is consistent with effects reported in later calibration work and is worth flagging when using MC Dropout in pipelines where the absolute confidence value matters.

Decomposition of total predictive uncertainty. Using the standard split $\mathbb{H}[\mathbb{E}_t p_t] = \mathbb{E}_t \mathbb{H}[p_t] + \text{MI}(y; \omega)$, where the second term is the BALD epistemic component [Houlsby et al., 2011], we find that on *correct* predictions 39% of the entropy is epistemic, against 37% on *wrong* predictions – but in absolute terms the epistemic mutual information on wrong predictions is 9 $\times$ larger (0.36 vs. 0.04 nats). Both components carry signal: aleatoric uncertainty inflates because the input is genuinely ambiguous, epistemic uncertainty inflates because individual stochastic networks disagree about what to predict.

What could be improved. A more faithful Bayesian-optimisation precision search would tighten the UCI numbers. Reproducing more than one Table 1 row (Boston, Energy, Kin8nm) would let us replicate the paper’s headline message that MC Dropout beats VI and PBP across a range of regression tasks. Running multiple seeds for the UCI experiment, rather than relying on splits alone, would also help. None of this changes the conclusion.

References

- Yarin Gal and Zoubin Ghahramani. Dropout as a Bayesian approximation: Representing model uncertainty in deep learning. In *ICML*, 2016.
- Neil Houlsby, Ferenc Huszár, Zoubin Ghahramani, and Máté Lengyel. Bayesian active learning for classification and preference learning. *arXiv:1112.5745*, 2011.
- Alex Graves. Practical variational inference for neural networks. In *NIPS*, 2011.
- José Miguel Hernández-Lobato and Ryan P. Adams. Probabilistic back-propagation for scalable learning of Bayesian neural networks. In *ICML*, 2015.
- Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1), 2014.
- Yann LeCun and Corinna Cortes. The MNIST database of handwritten digits, 1998.
- Han Xiao, Kashif Rasul, and Roland Vollgraf. Fashion-MNIST: a novel image dataset for benchmarking machine learning algorithms. *arXiv:1708.07747*, 2017.

A Additional figures

Architecture. Figure 3 shows the convolutional network used throughout the classification experiments.

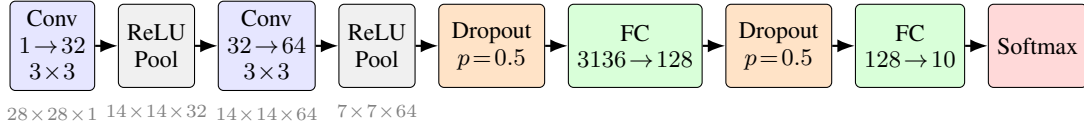


Figure 3: Classification network. Orange dropout blocks are the layers that remain active at test time when performing MC Dropout. Total parameters: 421 642.

Training curves and qualitative examples. Figure 4 shows the training loss and accuracy. Figure 5 shows the eight most confident and eight most uncertain MNIST test images under MC Dropout entropy; uncertain images include genuinely ambiguous digits.

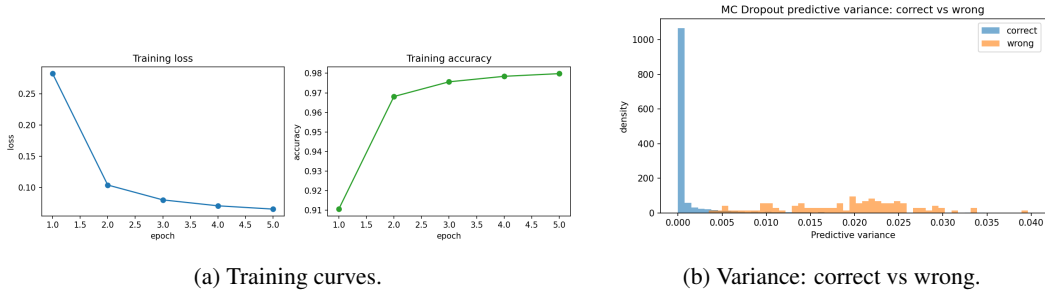


Figure 4: Training and the variance counterpart of Figure 2 (left).

Rotating-digit mean view, OOD histogram, decomposition, confusion, selective prediction.

B Hyperparameters and per-split results

C Accuracy / NLL vs. number of MC samples

D Implementation notes

MC Dropout in code. The whole machinery rests on a six-line helper that, after `model.eval()` disables every stochastic layer, re-enables *only* the dropout modules so that any batch-norm or other layer stays deterministic:

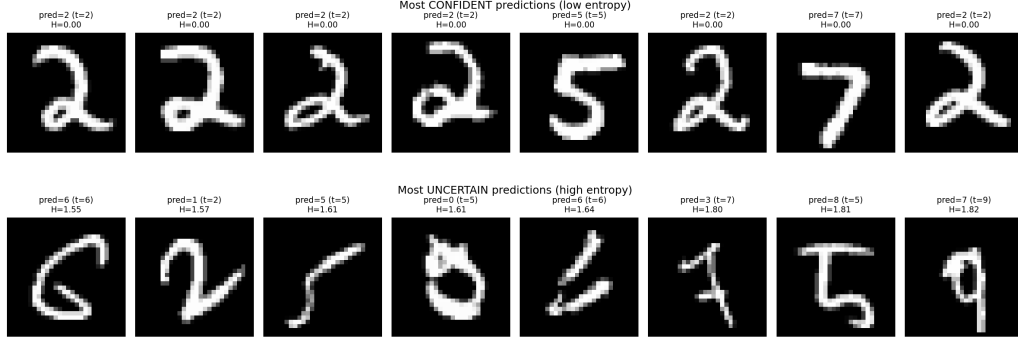


Figure 5: Top row: the eight most confident MC Dropout predictions (lowest predictive entropy) are clean, prototypical digits. Bottom row: the eight most uncertain predictions are visually ambiguous; for several of them the model’s top class differs from the true label.

Rotating digit (true label 1): MC Dropout uncertainty

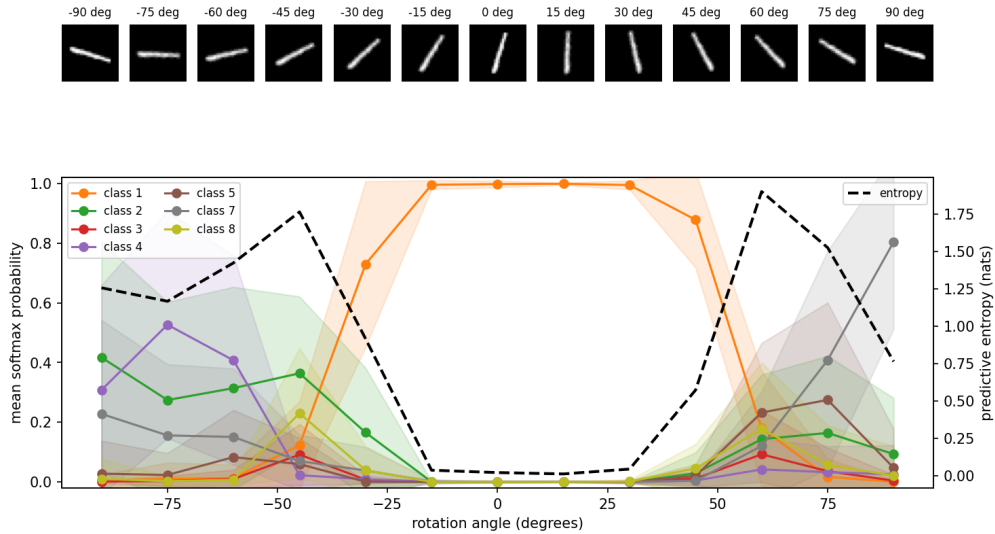


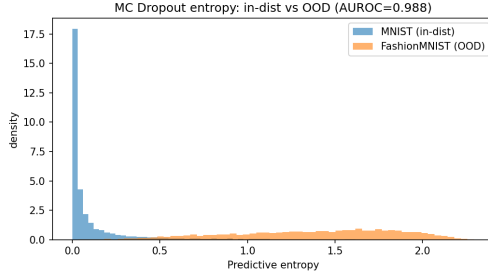
Figure 6: Rotating-digit experiment: mean softmax probability per class (coloured curves) $\pm$ one MC standard deviation, with the per-angle predictive entropy overlaid in black. Compare with the per-pass scatter in Figure 1.

```
def enable_dropout(model):
    for m in model.modules():
        if isinstance(m, torch.nn.Dropout):
            m.train()
```

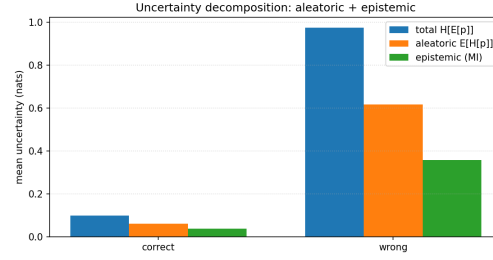
The remainder of `mc_dropout.mcdropout` is a thin wrapper that stacks T forward passes into a tensor of shape (T, N, C) and exposes `predictive_mean`, `predictive_entropy`, `expected_entropy`, `mutual_information`, `predictive_variance`, and `negative_log_likelihood`. The regression counterpart in `mc_dropout.uci` also implements equation (3) in a numerically stable form using `logsumexp`.

Reproducibility. Every script seeds Python, NumPy and PyTorch from a single `-seed` flag; deterministic data loaders are used. `requirements.txt` pins direct dependencies. End-to-end CPU runtime: training $\approx$ 1 min/epoch, all classification experiments together $\approx$ 12 min, UCI regression $\approx$ 4 min, multi-seed (three retrains) $\approx$ 6 min. The polished reproduction. `ipynb` re-executes every cell in the same order and saves the same figures into `figures/`.

What is in the repository.

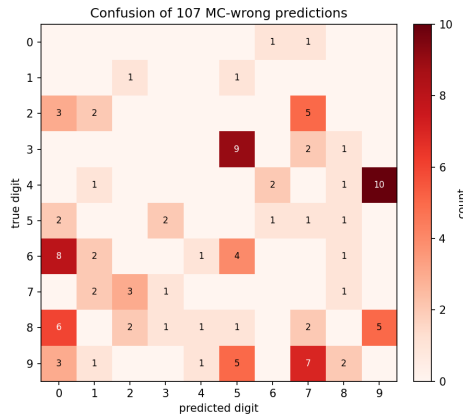


(a) OOD entropy histogram.

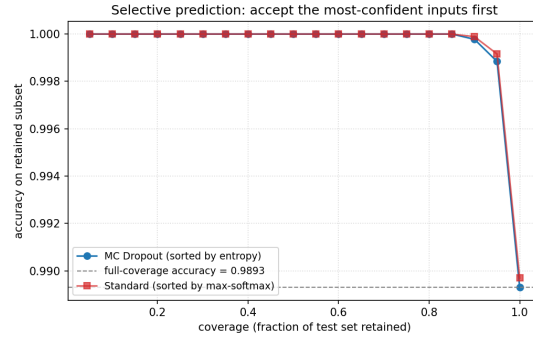


(b) Uncertainty decomposition.

Figure 7: Left: predictive entropy on MNIST vs. FashionMNIST under the same model (AUROC 0.988 with entropy alone). Right: total / aleatoric / epistemic components for correct and wrong MC predictions.



(a) Wrong-only confusion.



(b) Selective prediction.

Figure 8: Left: of the 107 MC-wrong predictions, the most frequent confusions are between visually similar digit pairs ($4 \leftrightarrow 9$, $3 \rightarrow 5$, $6 \rightarrow 0$, $9 \rightarrow 7$). Right: sorting the test set by entropy and accepting the most-confident fraction first, accuracy reaches 100% once roughly 5% of the most uncertain inputs are rejected.

- `src/mc_dropout/`: reusable library (model, data, MC inference, uncertainty metrics, training, UCI regression).
- `experiments/`: one numbered script per claim, each saving its figures to `figures/` and CSVs to `results/`. `08_extras.py` produces the calibration, rejection, decomposition, confusion and OOD-score-comparison figures in this appendix.
- `reproduction.ipynb`: self-contained narrative notebook covering Sections 1–8 of this report.
- `report.tex`, `neurips_2026.sty`, this PDF.

Use of AI tools

A large language model assistant (Anthropic Claude) was used as an editing helper during the writing of this report, mainly to suggest cleaner LaTeX phrasings and to flag awkward sentences. The conceptual structure of the report, the choice and interpretation of all experiments, and every numerical result are the author’s own work. The reproduction code was implemented and verified manually; the assistant was occasionally consulted to debug plotting issues and to format a small number of helper utilities. No AI tool generated the final claims or numbers presented here.

Work distribution

Single author. All design, implementation, analysis, and writing were carried out by Yasmine Gharsallah.

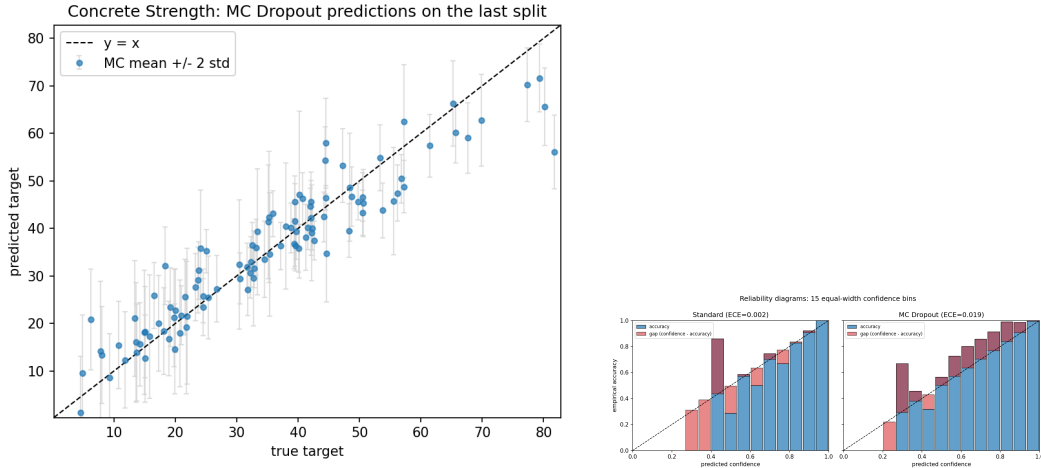


Figure 9: Left: MC Dropout regression on the last Concrete Strength split. Markers are the MC mean, error bars are ± 2 MC standard deviations; the dashed line is $y = x$. Right: reliability diagrams; standard predictions are extremely well-calibrated (ECE 0.002), MC Dropout is mildly under-confident (ECE 0.019).

Table 3: All hyperparameters used in this reproduction.

Component	Hyperparameter	Value
MNIST classifier (Sections 2–3)	architecture	Conv(1, 32) $\rightarrow$ Conv(32, 64) $\rightarrow$ FC(3136, 128) $\rightarrow$ FC(128, 10)
	dropout probability p	0.5 (before both FC layers)
	optimiser	Adam, lr 10^{-3} , batch 128
	weight decay λ	10^{-4}
	epochs	5
	MC samples T	30 (Sections 2–3), 100 (rotating digit)
UCI MLP (Section 2, Table 2)	architecture	FC(8, 50) $\rightarrow$ FC(50, 1), ReLU
	dropout probability p	0.05 (before both layers)
	optimiser	Adam, lr 10^{-3} , batch 32
	weight decay λ	10^{-4}
	epochs	400
	MC samples T	1000
	precision search	validation-LL grid over {0.05, 0.1, 0.15, 0.2, 0.25, 0.5, 1, 2}
	splits	5 random 90/10 splits

Table 4: Multi-seed MNIST results (3 seeds, 3 epochs each, $T = 30$).

seed	std. acc.	MC acc.	entropy (correct)	entropy (wrong)	ratio
0	0.9898	0.9893	0.1135	1.0096	8.89
1	0.9886	0.9883	0.1089	0.9941	9.13
2	0.9881	0.9877	0.1025	0.9308	9.08
mean	0.9888	0.9884	0.1083	0.9782	9.03
std	0.0007	0.0007	0.0045	0.0341	0.10

Table 5: UCI Concrete Strength per-split results.

split	τ	RMSE	log-likelihood
0	0.05	5.296	-3.079
1	0.05	6.262	-3.243
2	0.05	4.924	-3.025
3	0.05	5.592	-3.101
4	0.05	6.085	-3.231
mean	—	5.632	-3.136
std	—	0.494	0.086

Table 6: Accuracy and per-example NLL of the MC estimate as a function of the number of stochastic forward passes T on the MNIST test set.

T	1	2	3	5	10	20	30	50	75	100
accuracy	0.979	0.987	0.988	0.989	0.989	0.990	0.989	0.990	0.989	0.990
NLL	0.065	0.051	0.048	0.047	0.045	0.045	0.044	0.044	0.044	0.044

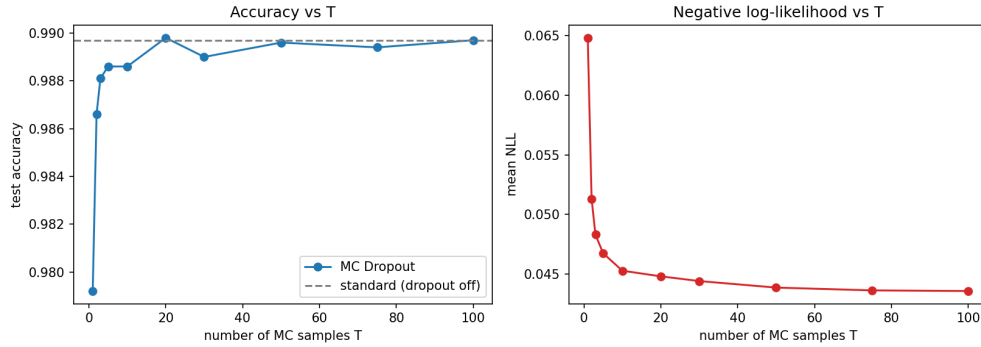


Figure 10: Accuracy (left) and per-example NLL (right) of MC Dropout as a function of T on the MNIST test set. The grey dashed line on the left panel is the standard (dropout off) accuracy. Both metrics are essentially flat from $T \geq 20$, consistent with the paper’s claim that small T is sufficient in practice.